Project 0: Getting Real

Preliminaries

> Fill in your name and email address.

Yuqi Su iyukisu@stu.pku.edu.cn

> If you have any preliminary comments on your submission, notes for the TAs, please give them here.

> Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, lecture notes, and course staff.

Booting Pintos

> A1: Put the screenshot of Pintos running example here.

```
root@ca2b3decf469:~# ls
pintos  toolchain
root@ca2b3decf469:~# cd pintos/src/threads/
root@ca2b3decf469:~/pintos/src/threads# make
cd build && make all
make[1]: Entering directory '/home/PKUOS/pintos/src/threads/build'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/PKUOS/pintos/src/threads/build'
root@ca2b3decf469:~/pintos/src/threads# cd build
root@ca2b3decf469:~/pintos/src/threads/build# pintos --
qemu-system-i386 -device isa-debug-exit -drive format=raw,media=disk,index=0,file=/tmp/L5DU7IUo5O.dsk -m 4 -net none -n
ographic -monitor null
Pintos hda1
Loading...........
Kernel command line:
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  157,081,600 loops/s.
Boot complete.
```

```
(base) PS D:\Desktop\pintos> docker run -it --rm --name pintos --mount type=bind,source=D:\Desktop\pintos,target=/home/
PKUOS/pintos pkuflyingpig/pintos bash
root@40df7f6c2fae:~# cd pintos/src/threads
root@40df7f6c2fae:~/pintos/src/threads# make
cd build && make all
make[1]: Entering directory '/home/PKUOS/pintos/src/threads/build'

make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/PKUOS/pintos/src/threads/build'
root@40df7f6c2fae:~/pintos/src/threads# cd build
root@40df7f6c2fae:~/pintos/src/threads/build# pintos --bochs --
squish-pty bochs -q
========================================================================
                      Bochs x86 Emulator 2.6.2
              Built from SVN snapshot on May 26, 2013
                Compiled on Mar  1 2022 at 16:09:16
========================================================================
00000000000i[     ] reading configuration from bochsrc.txt
00000000000e[     ] bochsrc.txt:8: 'user_shortcut' will be replaced by new 'keyboard' option.
00000000000i[     ] installing nogui module as the Bochs GUI
00000000000i[     ] using log file bochsout.txt
Pintos hda1
Loading...........
Kernel command line:
Pintos booting with 4,096 kB RAM...
383 pages available in kernel pool.
383 pages available in user pool.
Calibrating timer...  102,400 loops/s.
Boot complete.
```

Debugging

QUESTIONS: BIOS

> B1: What is the first instruction that gets executed?
> `jmp $0x3630, $0xf000e05b`

> B2: At which physical address is this instruction located?

```
0xffff0
((0xf000 << 4) + 0xfff0)
```

QUESTIONS: BOOTLOADER

> B3: How does the bootloader read disk sectors? In particular, what BIOS interrupt is used?

- By calling `read_sector` function.
- BIO interrupt: `int $0x13(0x7d1f)` .

> B4: How does the bootloader decides whether it successfully finds the Pintos kernel?

In `check_partition`, the bootloader checks:

- whether it is a Pintos kernel: `cmpb $0x20, %es:4(%si)`
- whether it is a bootable kernel: `cmpb $0x80, %es:(%si)`
  If both satisfied, bootloader thinks it finds the Pintos kernel.

> B5: What happens when the bootloader could not find the Pintos kernel?

It jumps to label `no_boot_partition`

- puts("Not found.")
- Notify BIOS has failed: `int $0x18`

> B6: At what point and how exactly does the bootloader transfer control to the Pintos kernel?

- Timing: This occurs at the end of the `load_kernel`, once all kernel sectors have been successfully read into memory.
- Mechanism:
  - The bootloader retrieves the kernel's entry point address from the ELF header and stores it in the memory location labeled `start`.
  - executes `ljmp *start`. This action sets the CPU's instruction pointer to the kernel's entry point, transferring control to the Pintos kernel.

QUESTIONS: KERNEL

> B7: At the entry of pintos_init(), what is the value of expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?
> `0x0`

```
(gdb) break pintos_init
Breakpoint 1 at 0xc00202b6: file ../../threads/init.c, line 80.
(gdb) c
Continuing.
The target architecture is assumed to be i386
=> 0xc00202b6 <pintos_init>:    push   %ebp

Breakpoint 1, pintos_init () at ../../threads/init.c:80
(gdb) p /x init_page_dir[pd_no(ptov(0))]
=> 0xc000efef:  int3
=> 0xc000efef:  int3
$1 = 0x0
```

> B8: When `palloc_get_page()` is called for the first time,
>
> > B8.1 what does the call stack look like?
> > `0# palloc_get_page`
> > `1# paging_init`
> > `2# pintos_init`
> > `3# start`

B8.2 what is the return value in hexadecimal format?
`0xc0101000`

B8.3 what is the value of expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?
`0x0`

```
(gdb) break palloc_get_page
Breakpoint 2 at 0xc0023213: file ../../threads/palloc.c, line 113.
(gdb) c
Continuing.
=> 0xc0023213 <palloc_get_page+6>:      sub    $0x8,%esp

Breakpoint 2, palloc_get_page (flags=(PAL_ASSERT | PAL_ZERO)) at ../../threads/palloc.c:113
(gdb) backtrace
#0  palloc_get_page (flags=(PAL_ASSERT | PAL_ZERO)) at ../../threads/palloc.c:113
#1  0xc00204a3 in paging_init () at ../../threads/init.c:224
#2  0xc002031b in pintos_init () at ../../threads/init.c:102
#3  0xc002013d in start () at ../../threads/start.S:180
(gdb) finish
Run till exit from #0  palloc_get_page (flags=(PAL_ASSERT | PAL_ZERO)) at ../../threads/palloc.c:113
=> 0xc00204a3 <paging_init+17>: add    $0x10,%esp
0xc00204a3 in paging_init () at ../../threads/init.c:224
Value returned is $2 = (void *) 0xc0101000
(gdb) p /x init_page_dir[pd_no(ptov(0))]
=> 0xc000ef8f:  int3
=> 0xc000ef8f:  int3
$3 = 0x0
```

B9: When palloc_get_page() is called for the third time,

B9.1 what does the call stack look like?
start()-pintos_init()-thread_start()-thread_create()-palloc_get_page()
`0# palloc_get_page`
`1# thread_createe`
`2# thread_start`
`3# pintos_init`
`4# start`

B9.2 what is the return value in hexadecimal format?
`0xc0103000`

B9.3 what is the value of expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?
`0x102027`

```
(gdb) c
Continuing.
=> 0xc0023213 <palloc_get_page+6>:      sub    $0x8,%esp

Breakpoint 2, palloc_get_page (flags=(PAL_ASSERT | PAL_ZERO)) at ../../threads/palloc.c:113
(gdb) c
Continuing.
=> 0xc0023213 <palloc_get_page+6>:      sub    $0x8,%esp

Breakpoint 2, palloc_get_page (flags=PAL_ZERO) at ../../threads/palloc.c:113
(gdb) backtrace
#0  palloc_get_page (flags=PAL_ZERO) at ../../threads/palloc.c:113
#1  0xc0020b7a in thread_create (name=0xc002e9c1 "idle", priority=0, function=0xc0020fa9 <idle>, aux=0xc000efbc) at ../../threads/thread.c:178
#2  0xc0020a6f in thread_start () at ../../threads/thread.c:111
#3  0xc0020334 in pintos_init () at ../../threads/init.c:121
#4  0xc002013d in start () at ../../threads/start.S:180
(gdb) finish
Run till exit from #0  palloc_get_page (flags=PAL_ZERO) at ../../threads/palloc.c:113
=> 0xc0020b7a <thread_create+55>:       add    $0x10,%esp
0xc0020b7a in thread_create (name=0xc002e9c1 "idle", priority=0, function=0xc0020fa9 <idle>, aux=0xc000efbc) at ../../threads/thread.c:178
Value returned is $4 = (void *) 0xc0103000
(gdb) p /x init_page_dir[pd_no(ptov(0))]
=> 0xc000ef4f:  int3
=> 0xc000ef4f:  int3
$5 = 0x102027
(gdb)
```

Kernel Monitor

C1: Put the screenshot of your kernel monitor running example here. (It should show how your kernel shell respond to `whoami`, `exit`, and `other input`.)

```
root@efd2af29d45c:~/pintos/src/threads/build# pintos --
qemu-system-i386 -device isa-debug-exit -drive format=raw,media=disk,index=0,file=/tmp/s6M39kHpU4.dsk -m 4 -net none -nographic -monitor null
Pintos hda1
Loading............
Kernel command line:
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  130,457,600 loops/s.
Boot complete.
PKUOS>hi
invalid command
PKUOS>
PKUOS>?????
invalid command
PKUOS>whoami
2400013053
PKUOS>qaq
invalid command
PKUOS>exit
Timer: 1688 ticks
Thread: 1662 idle ticks, 26 kernel ticks, 0 user ticks
Console: 379 characters output
Keyboard: 0 keys pressed
Powering off...
root@efd2af29d45c:~/pintos/src/threads/build#
```

C2: Explain how you read and write to the console for the kernel monitor.

- **Output:** I implemented the `tiny_shell` using `printf()` for output. `printf()` is a standard Pintos library function that eventually invokes low-level output routines (`vga_putc` and `serial_putc`) to display characters on the VGA screen and the serial port, respectively.
- **Input:** I used `input_getc()` from `devices/input.h` to read user input. This function fetches characters from a synchronized circular **buffer**. This buffer is populated asynchronously by the keyboard **interrupt handler** (`kbd_interrupt` in `devices/kbd.c`), which captures scan codes from hardware interrupts and translates them into ASCII characters. `input_getc()` blocks the execution until a character is available in the buffer.